T.C.

BALIKESİR ÜNİVERSİTESİ

MÜHENDİSLİK FAKÜLTESİ

BİLGİSAYAR MÜHENDİSLİĞİ

Kamera Görüntülerinden Araç Tespiti ve Park Uygunluğu Analizi için Yapay Zeka Tabanlı Sistem

BİTİRME PROJESİ

Hazırlayanlar

Kerem Yağız KARAKAŞ 202213709064

Ahmet EKŞİOĞLU 202113709025

Ezgi KIRNAPÇI 202113709041

Danışman

Prof. Dr. Selçuk KAVUT

Balıkesir 2026

İçindekiler

KISALTMALAR VE SİMGELER 5
ÖNSÖZ 7
ÖZET 8
1. GİRİŞ 9
1.1 Projenin Konusu ve Amacı 9
1.2 Neden Bu Problem Önemli? 9
1.3 Motivasyon 10
1.4 Sistemin Genel Yapısı 10
1.5 Araştırma Sorusu ve Hipotez 11
1.6 Projenin Kapsamı 11
2. Literatür Araştırması 13
2.1 Problem Nasıl Tanımlanıyor? 13
2.2 Tarihsel Gelişim: Derin Öğrenmeden Öncesi 13
2.3 Derin Öğrenme ile Gelen İki Ana Yaklaşım 14
2.4 Literatürdeki Önemli Çalışmalar 15
2.4.1 AlexNet ile Park Doluluk Tespiti (2015) 15
2.4.2 mAlexNet — Dağıtık Park Tespiti (Amato vd., 2017) 15
2.4.3 CarNet — Dilate CNN ile Park Tespiti (Nurullayev & Lee, 2019) 16
2.4.4 Geliştirilmiş MobileNetV3 + CBAM (Yuldashev vd., 2023) 16
2.4.5 APSD-OC — Otomatik Park Yeri Tespiti (Grbić & Koch, 2023) 16
2.4.6 PakLoc + PakSta — İş Yükünü %94 Azaltmak (2024) 17
2.4.7 YOLOv8/v9/v10/v11 Karşılaştırması (da Luz vd., 2024) 17
2.4.8 Gerçek Zamanlı Yasadışı Park Tespiti (Xie vd., 2017) 18
2.4.9 YOLOv7 ile Gerçek Zamanlı Park İzleme (2025) 18
2.5 Önemli Bir Sorun: Geçici Araç mı, Park Edilmiş Araç mı? 18
2.6 YOLO Versiyonları Karşılaştırması 19
2.7 Benchmark Veri Setleri 19
2.8 Sensör Tabanlı vs Kamera Tabanlı Sistemler 20
2.8.1 Parquery (İsviçre, 2014 — ETH Zürich Çıkışlı) 21
2.8.2 Quercus Technologies (İspanya, 25+ Yıl) 22
2.8.3 Metropolis (ABD, 2012) 22

2.8.4 ParkHub + Smarking (ABD) 22
2.8.5 IEM (Almanya) — Intelligent Parking Solutions 23
2.8.6 Bosch (Almanya) — Çok Katmanlı Park Çözümleri 23
2.8.7 Hikvision (Çin) — Büyük Ölçekli AI Kamera Sistemleri 23
2.8.8 Dahua Technology (Çin) — WizMind AI Platformu 24
2.8.9 Frogparking (Yeni Zelanda) — Hibrit Sensör Çözümü 24
2.9 Projenin Literatürdeki Yeri 25
2.10 Literatür Özeti Tablosu 26
3. Sistem Mimarisi 28
3.1 Genel Bakış 28
3.2 Bileşen 1: Araç Tespiti (VehicleDetector) 28
3.3 Bileşen 2: Park Alanı Etiketleme (ZoneAnnotator) 29
3.4 Bileşen 3: IoU Hesabı (ZoneLoader) 29
3.5 Bileşen 4: Karar Motoru 30
3.6 Bileşen 5: Sentetik Veri Üretici 30
3.7 Bileşen 6: YOLOv8 Fine-Tuning 30
3.8 Bileşen 7: Arayüz (MainWindow) 31
4. Kullanılan Teknolojiler 32
4.1 Programlama Dili: Python 3.12 32
4.2 Nesne Tespiti: YOLOv8 (Ultralytics) 32
4.3 Görüntü İşleme: OpenCV [15] 33
4.4 Derin Öğrenme Altyapısı: PyTorch 2.2.2 [16] 33
4.5 Görsel Arayüz: PyQt5 5.15+ 33
4.6 Model Değerlendirme: scikit-learn 1.3+ 33
4.7 Veri İşleme ve Görselleştirme 33
4.8 Teknoloji Özet Tablosu 33
5. Uygulama ve Bulgular 35
5.1 Geliştirme Süreci 35
5.2 Baseline Sistem Sonuçları 35
5.3 Sentetik Veri Üretimi 35
5.4 Fine-Tuning Sonuçları 35
5.5 Görsel Arayüz 36
5.6 IoU Tabanlı Doluluk Analizi 36

5.7 Mevcut Sistem Sınırlamaları 37
6. SONUÇ VE GELECEK ÇALIŞMALAR 38
6.1 Genel Değerlendirme 38
6.2 Literatürle Karşılaştırma 38
6.3 Karşılaşılan Zorluklar 38
KAYNAKLAR 39

KISALTMALAR VE SİMGELER

Kısaltma	Açılımı
AI	Artificial Intelligence (Yapay Zeka)
APSD-OC	Automatic Parking Space Detection and Occupancy Classification
COCO	Common Objects in Context
CNN	Convolutional Neural Network (Evrişimli Sinir Ağı)
CPU	Central Processing Unit (Merkezi İşlem Birimi)
CUDA	Compute Unified Device Architecture
DLL	Dynamic Link Library
FPS	Frames Per Second (Saniyedeki Kare Sayısı)
GPU	Graphics Processing Unit (Grafik İşlem Birimi)
IoU	Intersection over Union (Kesişim / Birleşim)
JSON	JavaScript Object Notation
mAP	Mean Average Precision (Ortalama Kesinlik)
NVIDIA	Graphics processing company
PyQt5	Python binding for Qt5 framework
ROI	Region of Interest (İlgi Bölgesi)
VRAM	Video Random Access Memory
YOLO	You Only Look Once
YOLOv8	You Only Look Once version 8

Kısaltma	Açılımı
YAML	Yet Another Markup Language

Simge	Açıklama
IoU	Intersection over Union değeri (0 ile 1 arasında)
P	Precision (Kesinlik): TP / (TP + FP)
R	Recall (Duyarlılık): TP / (TP + FN)
F1	F1-Score: 2 × (P × R) / (P + R)
mAP@0.5	IoU eşiği 0.5 alındığında ortalama hassasiyet
TP	True Positive (Doğru Pozitif)
TN	True Negative (Doğru Negatif)
FP	False Positive (Yanlış Pozitif)
FN	False Negative (Yanlış Negatif)
conf	Confidence score — modelin tahminine olan güven skoru (0-1)
px	Piksel

ÖNSÖZ

Bu çalışma, Balıkesir Üniversitesi Bilgisayar Mühendisliği Bölümü Bitirme Projesi kapsamında hazırlanmıştır. Proje; kamera görüntülerinden araç tespiti yapan ve park alanlarının doluluk durumunu gerçek zamanlı olarak analiz eden yapay zeka tabanlı bir sistem geliştirilmesi amacıyla yürütülmüştür.

Geliştirilen sistem, bilgisayar veya harici kameradan alınan görüntüler üzerinde derin öğrenme yöntemleriyle araç tespiti yapmakta ve park uygunluğunu gerçek zamanlı olarak analiz etmektedir. Çalışma boyunca YOLOv8 nesne tespit modeli, OpenCV görüntü işleme kütüphanesi, IoU tabanlı bölge analizi ve PyQt5 masaüstü arayüzü bir arada kullanılmıştır.

Proje danışmanımız Prof. Dr. Selçuk KAVUT'a yönlendirmeleri ve değerli geri bildirimleri için teşekkür ederiz.

ÖZET

Kentsel alanlarda araç sayısının hızla artması, park altyapısının yetersiz kalmasına ve park yönetiminin giderek daha karmaşık bir hal almasına yol açmaktadır. Mevcut sistemler park alanlarının doluluk durumunu ve yasak bölge ihlallerini gerçek zamanlı olarak izleyememekte; denetim büyük ölçüde manuel kontrole dayanmaktadır.

Bu projede, söz konusu soruna yapay zeka tabanlı otomatik bir çözüm geliştirilmesi hedeflenmiştir. Geliştirilen sistem üç temel bileşenden oluşmaktadır: YOLOv8 derin öğrenme modeli ile gerçek zamanlı araç tespiti, OpenCV tabanlı görüntü işleme ile park alanı analizi ve IoU (Intersection over Union) hesabına dayalı kural tabanlı karar mekanizması. Sistem bu bileşenlerin çıktısını birleştirerek "Park Edilebilir", "Park Alanı Dolu" ve "Park Edilemez" şeklinde üç sınıflı bir karar üretmektedir.

Proje kapsamında Microsoft COCO veri setiyle önceden eğitilmiş YOLOv8n modeli transfer learning yöntemiyle kullanılmıştır. Park senaryosuna özel eğitim için 1000 adet sentetik görüntü otomatik olarak üretilmiş ve YOLO formatında etiketlenmiştir. Park alanları, geliştirilen poligon tabanlı etiketleme aracı aracılığıyla tanımlanmıştır. PyQt5 çerçevesiyle gerçek zamanlı görselleştirme sağlayan bir masaüstü arayüzü geliştirilmiştir.

İlk testlerde YOLOv8n modeli araçları %80-95 güven skoru aralığında başarıyla tespit etmiştir. GPU desteğiyle 720p çözünürlükte 18-22 FPS gerçek zamanlı performans elde edilmiştir. Sistem bilgisayar kamerası veya harici kamera ile çalışabilmektedir.

Çalışmanın devam eden aşamalarında IoU pipeline entegrasyonu, MobileNetV2 ikinci doğrulama katmanı ve ByteTrack araç takip modülü eklenmesi planlanmaktadır. Performans değerlendirmesi Accuracy, Precision, Recall, F1-Score ve mAP metrikleriyle gerçekleştirilecektir.

1. GİRİŞ

1.1 Projenin Konusu ve Amacı

Dünya genelinde kentsel nüfus hızla artmaya devam etmekte ve buna paralel olarak şehirlerdeki araç sayısı da her geçen yıl yükselmektedir. Türkiye İstatistik Kurumu verilerine göre Türkiye'deki kayıtlı araç sayısı 2023 yılında 27 milyonu aşmıştır. Bu artış, zaten yetersiz olan park altyapısını daha da büyük bir baskı altına sokmaktadır.

Özellikle şehir merkezlerinde yasadışı park, günlük hayatın olağan bir parçası haline gelmiştir. Kaldırıma park edilen araçlar yayaların ve engellilerin geçişini engellemektedir. Çift sıra park edenler trafiği tıkamakta, kavşak köşelerine park edenler görüş açısını kapatarak kaza riskini artırmaktadır. Yangın çıkışlarının ve acil servis yollarının önünü kapatan araçlar ise can güvenliğini tehdit etmektedir.

Bu sorunla başa çıkmak için kullanılan mevcut yöntemler büyük ölçüde trafik polisinin ya da güvenlik görevlilerinin sahaya çıkıp araçları tek tek kontrol etmesine dayanmaktadır. Bu yaklaşımın üç temel sorunu bulunmaktadır: birincisi, yalnızca görevlinin orada bulunduğu anlarda etkili olmaktadır; ikincisi, çok sayıda personel gerektirdiği için maliyetlidir; üçüncüsü, insan hatasına açık olduğundan tutarsız uygulamaya yol açabilmektedir.

Bu projede söz konusu soruna tamamen otomatik ve ölçeklenebilir bir çözüm geliştirilmesi hedeflenmiştir. Geliştirilen sistem; bilgisayar kamerası, telefon kamerası veya harici bir kameradan alınan görüntüleri yapay zeka ile analiz ederek araçları tespit etmekte ve aracın bulunduğu konumun park kurallarına uygun olup olmadığını gerçek zamanlı olarak değerlendirmektedir.

1.2 Neden Bu Problem Önemli?

Park sorunu yüzeysel bakıldığında basit bir "düzensizlik" meselesi gibi görünebilir. Oysa etkisi çok daha derine inmektedir.

Trafik akışı üzerindeki etkisi: Çift sıra ya da yasak bölgeye park, trafik akışını bozarak ortalama seyahat sürelerini ciddi biçimde uzatmaktadır. Texas A&M Ulaştırma Enstitüsü'nün araştırmalarına göre trafik sıkışıklığı ABD'de yılda 87 milyar dolarlık kayba yol açmaktadır; bu kaybın önemli bir bölümü yasadışı park kaynaklı darboğazlardan kaynaklanmaktadır.

Acil müdahale gecikmeleri: İtfaiye ya da ambulans araçlarının yasadışı park nedeniyle geçemediği senaryolar teorik değildir. Araştırmalar, acil servislerin kentsel alanlarda ortalama müdahale süresinin yasadışı park nedeniyle %15-20 oranında uzayabildiğini göstermektedir.

Ekonomik kayıp: Yasadışı park cezaları ve bu cezaların tahsil edilememesi büyük bir sorun oluşturmaktadır. İngiltere'de her yıl tahsil edilemeyen park cezalarının toplamı yüz milyonlarca pound düzeyinde seyretmektedir. Otomatik tespit sistemleri bu kayıpları ciddi ölçüde azaltabilir.

Çevre etkisi: Araştırmalar, park yeri arayan araçların şehir trafiğinin yaklaşık %30'unu oluşturduğunu ve bu boşta dolaşmanın gereksiz yakıt tüketimine ve emisyona yol açtığını ortaya koymaktadır. Doluluk bilgisinin anlık paylaşılması bu problemi azaltabilir.

1.3 Motivasyon

Bu projenin gerçekleştirilmesinin teknik ve ekonomik gerekçeleri bulunmaktadır:

Pazar büyük ve büyümektedir: Küresel akıllı park sistemleri pazarının 2024 yılında 7.85 milyar dolar büyüklüğünde olduğu tahmin edilmektedir. 2035'e kadar yılda yaklaşık %17-18 büyümesi öngörülmekte ve pazar değerinin 47 milyar dolara ulaşması beklenmektedir [1].

Teknoloji artık buna hazırdır: 2015 yılında ilk YOLO modeli yayınlandığında gerçek zamanlı nesne tespiti yalnızca yüksek uçlu GPU'larda mümkündü. Bugün YOLOv8, orta sınıf bir dizüstü bilgisayarda 30+ FPS ile çalışabilmektedir. Hesaplama maliyeti önemli ölçüde düşmüştür.

Mevcut sistemler pahalı ve kırılgandır: Zemine gömülen manyetik sensörler veya döngü bobinleri yüksek kurulum maliyeti gerektirmektedir. Yol yüzeyi değiştiğinde ya da sensör arızalandığında sistemin tamamı devre dışı kalmaktadır. Kamera tabanlı bir yaklaşım aynı bilgiyi çok daha esnek ve düşük maliyetle üretebilmektedir.

Elektrikli araçlar yeni bir problem oluşturmaktadır: Manyetik sensörler, araçların kütlesi nedeniyle ortaya çıkan manyetik alan bozulmasını okuyarak çalışmaktadır. Ancak elektrikli araçlar hafif yapıları ve gövde tasarımları nedeniyle manyetik alana çok az etki etmektedir. Bu durum, yalnızca manyetometreye dayanan sistemleri giderek daha yetersiz kılmaktadır. Kamera tabanlı sistemler bu kısıtı yaşamamaktadır.

Transfer learning ile az veriyle güçlü model oluşturulabilmektedir: COCO gibi devasa veri setleri üzerinde önceden eğitilmiş modeller, transfer learning yöntemiyle ilgili probleme uyarlanabilmektedir. Bu yaklaşım, akademik bir proje için de uygulanabilir hale gelmektedir.

1.4 Sistemin Genel Yapısı

Geliştirilen sistem üç ana bileşenden oluşmakta ve bu bileşenler sıralı bir pipeline şeklinde çalışmaktadır:

Bileşen 1 — Araç Tespiti (YOLOv8) Kameradan gelen görüntü üzerinde araçlar tespit edilmektedir. Sistem araba, otobüs, kamyon ve motosiklet sınıflarını ayrı ayrı tanıyabilmektedir. Her tespit için bounding box koordinatları ve güven skoru hesaplanmaktadır. Microsoft COCO veri setiyle önceden eğitilmiş YOLOv8n modeli kullanılmaktadır [2].

Bileşen 2 — Park Alanı Analizi (OpenCV + IoU) Park alanları ve yasak bölgeler sisteme poligon çizilerek bir kez tanıtılmaktadır. Ardından araç tespitinden gelen bounding box ile park alanı poligonu arasındaki örtüşme oranı hesaplanmaktadır. Bu hesaplama IoU (Intersection over Union) yöntemiyle gerçekleştirilmektedir. IoU > 0.3 eşiğini geçen araçlar için ilgili park yeri "dolu" kabul edilmektedir.

Bileşen 3 — Karar Mekanizması (Kural Motoru) IoU analizi sonucuna ve park alanının özelliğine bağlı olarak üç sınıftan biri üretilmektedir:

- Park Edilebilir: Araç yok, alan uygun

- Park Alanı Dolu: Alan dolu, kurallara uygun araç mevcut
- Park Edilemez: Araç yasak bölgedeyse ya da uygunsuz konumdaysa

1.5 Araştırma Sorusu ve Hipotez

Temel araştırma sorusu: Kentsel alanlarda kamera görüntülerinden hareketle, bir aracın bulunduğu konumda park etmesinin uygun olup olmadığı; derin öğrenme tabanlı nesne tespiti ve görüntü işleme tekniklerinin birlikte kullanımıyla, insan müdahalesine ihtiyaç duyulmaksızın, gerçek zamanlı ve güvenilir biçimde otomatik olarak belirlenebilir mi?

Alt sorular:

1. YOLOv8 tabanlı nesne tespit modeli, farklı ışık koşulları ve trafik yoğunluğu senaryolarında araçları ne ölçüde doğru ve tutarlı biçimde tespit edebilmektedir?
2. Geliştirilen park uygunluğu sınıflandırıcısı, farklı kamera açılarında %85 ve üzeri doğruluk değerine ulaşabilmekte midir?
3. IoU analizi tek başına park uygunluğunu belirlemek için yeterli midir, yoksa ek bir sınıflandırma katmanına ihtiyaç duyulmakta mıdır?
4. Sentetik veriyle eğitilen model gerçek kamera görüntülerine ne kadar iyi genelleyebilmektedir?

Hipotez: YOLOv8 tabanlı nesne tespiti ile poligon tabanlı park alanı analizinin kural motoru aracılığıyla birleştirilmesinin; park uygunluğunu "Park Edilebilir", "Park Edilemez" ve "Park Alanı Dolu" şeklinde üç sınıfta, %85 ve üzeri doğrulukla, gerçek zamanlı olarak sınıflandırmak için yeterli olduğu öngörülmektedir.

1.6 Projenin Kapsamı

Kapsam içinde:

- Sabit kamera görüntüsünden araç tespiti (araba, otobüs, kamyon, motosiklet)
- Poligon tabanlı park alanı ve yasak bölge tanımlama aracı
- IoU tabanlı doluluk analizi
- 3 sınıflı park uygunluğu kararı
- Sentetik veri üretimi ve YOLOv8 fine-tuning
- Gerçek zamanlı grafik arayüz (PyQt5)
- Performans metrikleri: Accuracy, Precision, Recall, F1-Score, mAP

Kapsam dışında:

- Plaka tanıma ve araç kimliği
- Otomatik ceza kesme veya bildirim sistemi
- Mobil uygulama
- Gece görüş kamerası entegrasyonu
- Araç takibi (tracking)

2. Literatür Araştırması

Hocamızın yönlendirmesiyle bu alanda sıfırdan bir şey icat etmeye çalışmak yerine, literatürde neyin yapıldığını iyi anlayıp onun üzerine inşa etmeyi hedefledik. Bu bölümde park yeri tespit sistemleri konusunda şimdiye kadar yapılmış akademik çalışmalar, kullanılan veri setleri, izlenen yöntemler ve piyasadaki ticari ürünler detaylıca incelendi.

2.1 Problem Nasıl Tanımlanıyor?

Literatür bu problemi genellikle üç alt göreve bölüyor:

1. Park yeri konumu tespiti — Görüntüde hangi bölgeler park yeri? (Otomatik ya da manuel olarak sınırlar belirlenir.)
2. Doluluk sınıflandırması — Her park yeri dolu mu, boş mu? (İkili sınıflandırma)
3. Araç sayımı — Toplam kaç araç var? (Daha basit versiyon)

Bu üç görev, akıllı park sistemlerinin çekirdek bileşenlerini oluşturmaktadır. Sürücü yönlendirme ve otomatik yönetim gibi uygulamalar bu temel üzerine inşa edilmektedir. Geliştirilen projede bu üç görevin tamamı ele alınmaktadır: araç tespiti yapılmakta, park alanı sınırları tanımlanmakta ve doluluk kararı verilmektedir.

2.2 Tarihsel Gelişim: Derin Öğrenmeden Öncesi

Derin öğrenme yaygınlaşmadan önce sistemler üç temel tekniğe dayanıyordu:

a) Arka plan çıkarma (Background Subtraction): Kamera sabit olduğunda "boş park yeri" görüntüsü referans olarak alınır. Sonraki karelerde bu referanstan farklılaşan piksel bölgeleri araç olarak kabul edilir. Yöntemin mantığı basit ve anlaşılır, ama ışık değişimlerine son derece hassas. Sabah ile öğleden sonrasının farklı gün ışığı açısı bile sistemi yanıltabiliyor. Gölgeler, yansımalar ve hava koşulları bu yöntemi pratikte çok kırılgan yapıyor.

b) HOG + SVM: Her görüntüden kenar ve yoğunluk bilgisi çıkarılıyor (HOG — Histogram of Oriented Gradients), ardından bu özellikler bir makine öğrenmesi sınıflandırıcısı olan SVM'e veriliyor. HOG el yapımı bir özellik çıkarıcı olduğu için farklı açı ve ışık koşullarında tutarsız davranıyor. Orta düzeyde doğruluk sunsa da büyük ölçekli sistemler için yavaş. Buradaki SVM, derin öğrenme değil; insanın hangi özelliklere bakılacağını belirlediği daha klasik bir makine öğrenmesi yöntemidir.

c) Çizgi Tespiti (Hough Transform): Park çizgileri Hough dönüşümü algoritmasıyla tespit ediliyor, her bölgenin dolu/boş olduğu kontrol ediliyor. Kontrollü ortamlarda iyi çalışıyor ama çizgiler aşınmış, kirli, karlı ya da gölgeli olduğunda tamamen başarısız oluyor. Gerçek dünya park alanlarında bu koşullar son derece yaygın. Hough dönüşümü, görüntüdeki kenarları kullanarak düz çizgileri bulmaya çalışan bir yöntemdir.

Bu üç yöntemin ortak sorunu: kontrollü, sabit koşullarda makul sonuçlar veriyorlar ama gerçek dünya senaryolarına geçildiğinde performansları ciddi ölçüde düşüyor.

2.3 Derin Öğrenme ile Gelen İki Ana Yaklaşım

Derin öğrenme bu alana girince iki farklı paradigma ortaya çıktı. İkisi arasındaki seçim, sistemin tüm mimarisini belirliyor.

Yaklaşım A — Bölge Sınıflandırma (Patch-Based Classification)

Bu yaklaşımda her park yerine karşılık gelen küçük bir görüntü (patch) kırpılır ve bir CNN ile "dolu" veya "boş" olarak sınıflandırılır.

Tam park alanı görüntüsü

↓

[Manuel/Otomatik bbox tespiti]

↓

Her slot için patch kırpma

↓

CNN → Dolu / Boş

Bu yaklaşım PKLot ve CNRPark gibi veri setleriyle çok iyi sonuçlar verdi. Ama kritik bir kısıtı var: kamera açısı değiştiğinde ya da kamera söküldüğünde tüm park yeri koordinatlarının yeniden etiketlenmesi gerekiyor. 200 park yerli 5 ayrı parkı yönetmek durumundaki bir operatör için bu, ciddi bir iş yükü demek.

Bu yaklaşımın sınırları:

- Her kamera değişikliğinde yeniden etiketleme
- Farklı açılara genelleme zor
- Eğitim için büyük miktarda etiketli veri gerekiyor
- Sadece dolu/boş sınıflandırması yapıyor; ihlal türü belirlenemiyor

Yaklaşım B — Nesne Tespiti + IoU (Object Detection-Based)

Nesne dedektörü tüm görüntüyü işler; tespit edilen araçların konumları ile önceden bilinen park yeri sınırları IoU (Intersection over Union) hesaplanarak karşılaştırılır.

Kamera Görüntüsü

↓

[YOLOv8 ile Araç Tespiti]

↓

[Park Yeri Poligonlarıyla IoU Hesabı]

↓

[IoU > 0.5 → Dolu | IoU < 0.5 → Boş]

↓

[Doluluk Kararı + İhlal Tespiti]

Bu yaklaşım daha esnek çünkü park yeri konumları bir kez tanımlanıyor, sonraki her karedeki araç tespitleri otomatik karşılaştırılıyor. Ayrıca yasak bölge poligonu da tanımlanabiliyor; böylece ihlal tespiti de mümkün oluyor.

Geliştirilen proje bu ikinci yaklaşımı uygulamaktadır: YOLOv8 ile araç tespiti yapılmakta, park alanları fareyle tıklanarak poligon olarak tanımlanmakta, ardından IoU analizi ile doluluk kararı verilmektedir.

2.4 Literatürdeki Önemli Çalışmalar

2.4.1 AlexNet ile Park Doluluk Tespiti (2015)

Problem: PKLot veri setinin yayınlanmasıyla birlikte bu veri seti üzerinde AlexNet mimarisi uygulandı [5]. Çalışma, o dönem için oldukça güçlü sonuçlar verdi.

Nasıl yaptılar: Her park slotunu küçük bir görüntü olarak kestiler, AlexNet'e "dolu" veya "boş" olarak sınıflandırdılar. Güneşli havada %99'a yakın, yağmurlu havada %95 civarı doğruluk elde ettiler.

Önemi: Bu çalışma, derin öğrenmenin park tespitinde klasik yöntemlere kıyasla ne kadar üstün olduğunu ilk kez net biçimde gösterdi ve PKLot veri setini alandaki standart benchmark haline getirdi.

Kısıtı: AlexNet büyük bir model; kamerada çalıştırmak için çok ağır.

2.4.2 mAlexNet — Dağıtık Park Tespiti (Amato vd., 2017)

Problem: AlexNet iyi sonuçlar veriyordu ama her park kamerasında bir sunucu çalıştırmak mümkün değil. Daha hafif bir model gerekiyordu.

Nasıl yaptılar: Amato ve ekibi, AlexNet'e kıyasla üç kat daha küçük özel bir CNN tasarladı — buna mAlexNet dediler [3]. Aynı zamanda CNRPark veri setini yayınladılar. Model, kameranın kendi içinde çalışabilecek kadar hafif tasarlandı.

Sonuçlar: AUC (Alan Under Curve) değerleri 0.92-0.99 arasında ölçüldü. AlexNet'in yaklaşık yarısı büyüklüğünde bir modelle benzer doğruluk elde ettiler.

Önemi: Akıllı kamerada çalışan ilk ciddi park tespiti sistemi. "Edge computing" yaklaşımının bu alandaki öncüsü.

Dezavantajı: Modelin daha küçük ve hafif olacak şekilde tasarlanması, özellikle karmaşık sahnelerde ve farklı çevresel koşullarda temsil gücünü sınırlayarak doğrulukta düşüşe neden olabilmektedir.

2.4.3 CarNet — Dilate CNN ile Park Tespiti (Nurullayev & Lee, 2019)

Problem: Standart CNN'ler küçük nesneleri kaçırma eğiliminde [4]. Park yerleri de sahnede görece küçük nesneler.

Nasıl yaptılar: Dilate edilmiş evrişimli sinir ağları (dilated convolution) kullandılar. Bu teknik, filtrenin "boşluklu" hareket etmesini sağlıyor; böylece daha büyük bir alanı daha az hesapla görüyor. Her park slotu için 54×32 piksellik RGB görüntü girişi kullandılar.

Sonuçlar: Hem PKLot hem CNRPark-EXT üzerinde AlexNet ve mAlexNet'i geride bıraktılar. Ortalama doğruluk %97'nin üzerine çıktı.

Önemi: Dilated convolution'ın park tespitindeki etkinliğini gösterdi; küçük nesne tespitinde önemli bir adım oldu.

Dezavantajı: Bu yaklaşım daha geniş bağlam yakalamayı sağlasa da bazı durumlarda “gridding artefact” adı verilen ve detay kaybına yol açan yapısal bozulmalara neden olabilmektedir.

2.4.4 Geliştirilmiş MobileNetV3 + CBAM (Yuldashev vd., 2023)

Problem: Yüksek doğruluklu modeller ağır; düşük donanımlı sistemlerde çalışmıyor. Hem hafif hem doğru bir model gerekiyor [8].

Nasıl yaptılar: MobileNetV3 mimarisini üç modifikasyonla geliştirdiler:

1. ReLU6 aktivasyonu yerine Leaky-ReLU6 kullandılar — negatif değerleri tamamen kesmek yerine küçük bir sızıntı bıraktılar.
2. Squeeze-Excitation (SE) modülü yerine CBAM (Convolutional Block Attention Module) kullandılar — hem kanal hem de uzamsal dikkat mekanizması eklendi.
3. Standart depthwise separable convolution yerine Blueprint Separable Convolution (BSConv) kullandılar — daha az parametre, benzer performans.

Sonuçlar: PKLot alt setlerinde %99.68, CNRPark-EXT'de %97.69 doğruluk. Önceki en iyi model olan CarNet'in %97.03'ünü geçtiler. Bu, 2023 itibarıyla literatürdeki en yüksek sınıflandırma doğruluklarından biri.

Önemi: Hafif ama yüksek performanslı bir model isteyenler için güçlü bir referans. Dikkat mekanizmalarının park tespitindeki etkinliğini kanıtladı.

2.4.5 APSD-OC — Otomatik Park Yeri Tespiti (Grbić & Koch, 2023)

Problem: Tüm çalışmalarda park yeri koordinatları manuel olarak etiketleniyor [9]. Bu, büyük ölçekli sistemlerde ciddi iş yükü yaratıyor. Kamera değişince her şeyi yeniden yapmak gerekiyor.

Nasıl yaptılar: Tamamen otomatik bir iki aşamalı sistem geliştirdiler:

- 1. Aşama (Park Yeri Tespiti): YOLOv5 tüm araçları tespit ediyor → her tespitinin merkezi bir homografi matrisi aracılığıyla kuş bakışı görünüme dönüştürülüyor → bu noktalar kümelenerek park yeri sınırları belirleniyor.

- 2. Aşama (Doluluk Sınıflandırması): Belirlenen her park yeri ResNet34 tabanlı bir sınıflandırıcıya veriliyor → dolu/boş kararı üretiliyor.

Sonuçlar: PKLot ve CNRPark üzerinde yüksek doğruluk elde ettiler. Ama asıl önemli sonuç şu: insan iş yükü önemli ölçüde azaldı.

Önemi: Manuel etiketleme sorununa ilk sistematik çözüm. Geliştirilen sistemde park alanları hâlâ manuel olarak çizilmektedir. Bu çalışma, ileride otomatik hale getirmek için bir yol haritası niteliği taşımaktadır.

Kısıtı: Sistem, homografi dönüşümüne dayandığı için kamera açısı ve kalibrasyon hatalarına oldukça duyarlıdır; bu durum park yeri tespitinde hatalara yol açabilmektedir.

2.4.6 PakLoc + PakSta — İş Yükünü %94 Azaltmak (2024)

Problem: Manuel etiketleme problemi hâlâ çözülmüş sayılmıyor. APSD-OC otomasyonu getirdi ama pratik iş yükü azalması ne kadar?

Nasıl yaptılar: PakLoc (Park Locator) ve PakSta (Park Status) adında iki ayrı sistem geliştirdiler. PakLoc, park alanlarını otomatik tespit ediyor; PakSta ise doluluk durumunu belirliyor. PKLot veri seti üzerinde model bileşenlerinin katkısını analiz etmek için çalışma yaptılar.

Sonuçlar: İnsan iş yükünü %94.25 oranında azalttıklarını kanıtladılar. Bu rakam, büyük ölçekli sistemler için son derece anlamlı.

Önemi: "Otomasyonun gerçekten ne kadar iş yükü azalttığını" sayısal olarak ortaya koyan ilk çalışma.

Kısıtı: Elde edilen yüksek iş yükü azaltma oranı, kullanılan veri setine özgü olabilir ve farklı ortamlarda aynı seviyede performans garanti edilemeyebilir.

2.4.7 YOLOv8/v9/v10/v11 Karşılaştırması (da Luz vd., 2024)

Problem: YOLO'nun hangi versiyonu park tespiti için en uygun [7]? Ve pixel bazlı ROI seçimi performansı nasıl etkiliyor?

Nasıl yaptılar: 3.484 görüntüden oluşan kendi veri setlerini oluşturdular ve YOLOv8, YOLOv9, YOLOv10, YOLOv11 modellerini karşılaştırdılar. Bunun yanı sıra pixel bazlı ROI (Region of Interest) seçimi adlı bir post-processing tekniği önerdiler: tespit sonrası her park slotunun piksel düzeyinde doluluk oranı hesaplanıyor.

Sonuçlar: YOLOv9e, COCO veri setinde 55.6 mAP ile en yüksek tespit doğruluğunu verdi. Pixel bazlı ROI tekniği eklendikten sonra %99.68 balanced accuracy elde ettiler. Edge cihazlarda da test ettiler.

Önemi: Bu çalışma, geliştirilen projeye en yakın referans niteliğindedir. Aynı yaklaşım (YOLO + ROI analizi) temel alınmış; ilerleyen aşamalarda MobileNetV2 doğrulama katmanı eklenmesi planlanmaktadır.

2.4.8 Gerçek Zamanlı Yasadışı Park Tespiti (Xie vd., 2017)

Problem: Yasadışı park tespiti için mevcut sistemlerin büyük çoğunluğu yavaş çalışıyor ya da gerçek zamanlı değil [13].

Nasıl yaptılar: SSD (Single Shot MultiBox Detector) algoritmasını optimize ettiler. SSD'nin aspect ratio (en-boy oranı) parametrelerini araç tespit için ayarladılar. Tespit edilen araçlar için konum takibi yapıp ihlal süresini de hesapladılar.

Sonuçlar: %99 doğruluk, 25 FPS gerçek zamanlı çalışma.

Önemi: Yasadışı park tespiti alanında o dönem için en yüksek doğruluk elde edilmiştir. Geliştirilen projede de benzer şekilde yasak bölge altyapısı kurulmuştur; temel fark olarak SSD yerine daha güncel YOLOv8 modeli tercih edilmiştir.

Dezavantajı: Pixel tabanlı ROI yaklaşımı doğruluğu artırsa da ek bir işlem adımı gerektirdiğinden sistemin genel gecikme süresini artırabilmektedir.

2.4.9 YOLOv7 ile Gerçek Zamanlı Park İzleme (2025)

Problem: Araç tespiti güzel çalışıyor, peki bu bilgiyi nasıl yönetim sistemine entegre ederiz?

Nasıl yaptılar: YOLOv7'yi bir web arayüzüne bağladılar. Flask (Python web framework), OpenCV ve SQLite kullanarak gerçek zamanlı veri kaydı yapan bir sistem kurdu. Araç tespiti sonuçları veritabanına işleniyor, web panelinden görüntülenebiliyor.

Sonuçlar: Farklı çevre koşullarında ortalama %94.23 doğruluk. Sistem gerçek zamanlı çalışıyor.

Önemi: Sadece tespit değil, yönetim entegrasyonu da başarılı. Geliştirilen sistem şu aşamada masaüstü arayüzüne odaklanmaktadır; ilerleyen aşamada veritabanı entegrasyonu için bu çalışma referans alınabilir.

2.5 Önemli Bir Sorun: Geçici Araç mı, Park Edilmiş Araç mı?

Literatürün çoğu zaman atlayıp geçtiği ama pratikte son derece kritik bir problem var: park alanından sadece geçen bir araç da nesne dedektörü tarafından tespit edilebilir ve sistemi yanlış "dolu" kararına yönlendirebilir.

Bu problemi çözmek için araç takibi (tracking) kullanılıyor. Ardışık karelerde aynı araç takip ediliyor:

- Hareketsiz park edilmiş bir araç → ardışık karelerde aynı konumda, sürekli yüksek IoU
- Geçen bir araç → birkaç kare sonra sahneyi terk ediyor, düşük IoU tutarlılığı

Bu alanda öne çıkan algoritmalar:

SORT: Kalman filtresi + Macar algoritması. Hızlı ama örtüşme durumlarında araç kimliğini kaybediyor. Basit senaryolar için yeterli.

DeepSORT: SORT'a derin öğrenme tabanlı görsel benzerlik özelliği ekledi. Her araca görsel bir "imza" çıkarılıyor; hem konum hem de görünüş bilgisiyle eşleştirme yapılıyor. Örtüşen araçlarda daha güvenilir.

ByteTrack (ECCV 2022): Önemli bir yenilik getirdi: düşük güven skorlu tespitleri de işleme dahil ediyor [10]. Standart yaklaşımlar bu tespitleri atar; oysa örtülmüş bir araç çoğunlukla düşük güven skoru alır ama tamamen arka plan değildir. ByteTrack bu bilgiyi ikinci bir eşleştirme adımında kullanarak ID kayıplarını ciddi ölçüde azaltıyor.

Hedeflediğimiz kombinasyon: Park ihlali tespiti için literatür YOLOv8 + ByteTrack veya YOLOv8 + DeepSORT kombinasyonlarını öneriyor. Bizim sistemimizde henüz tracking yok; bu ilerleyen haftalarda eklenmesi planlanan bir özellik.

2.6 YOLO Versiyonları Karşılaştırması

Model	mAP (COCO)	Hız	Küçük Nesne	Park Tespitine Uygunluk
YOLOv8n	~37.3	En hızlı	Zayıf	Edge cihazlar, düşük GPU
YOLOv8m	~50.2	Orta	Orta	Standart GPU
YOLOv9e	55.6	Yavaş	İyi	Yüksek doğruluk gerektiğinde
YOLOv11n	~39.5	En hızlı	Orta+	Edge + doğruluk dengesi

Projede YOLOv8n kullanılmaktadır. GTX 1050 Ti Mobile gibi sınırlı bir GPU'da gerçek zamanlı çalışabilmesi için hız kritik. YOLOv8n bu modeller arasında en düşük hesaplama yüküne sahip ve gerçek zamanlı tespit için en uygun seçenek. Daha yüksek doğruluk gerekirse ilerleyen aşamada YOLOv8s veya YOLOv8m'e geçilebilir.

2.7 Benchmark Veri Setleri

Literatürdeki neredeyse tüm çalışmalar üç ana veri seti üzerinde test yapıyor. Bu veri setleri alandaki standart kıyaslama araçları haline geldi.

PKLot (UFPR, Brezilya, 2015)

Üç farklı park alanından (PUCPR, UFPR04, UFPR05) çekilmiş 12.417 tam park alanı görüntüsü ve yaklaşık 695.900 etiketli park yeri örneği içeriyor. Güneşli, bulutlu ve yağmurlu olmak üzere üç farklı hava koşulunu kapsıyor. Her park yeri hem döndürülmüş dikdörtgen hem de kontur formatında XML olarak etiketlenmiş. Alandaki standart benchmark veri seti olma özelliğini hâlâ koruyor.

Neden önemli: Bu veri seti üzerinde çalışan model, standart test koşullarında diğer çalışmalarla doğrudan karşılaştırılabiliyor. Doğruluk iddialarının güvenilir olması için PKLot üzerinde test şart.

CNRPark-EXT (CNR Pisa, İtalya, 2017)

9 farklı kamera açısından çekilmiş yaklaşık 150.000 etiketli patch görüntüsü içeriyor. 164 park slotunu farklı mevsimler ve koşullar altında kapsıyor. Gölgeler, kısmi örtülmeler, farklı mevsimler ve yılın farklı saatleri mevcut. Generalizasyon testi için daha zorlu bir veri seti.

Neden önemli: Farklı kameralar ve farklı koşullar modelin genelleme kapasitesini ölçüyor. Sadece bir veri setinde iyi sonuç vermek yeterli değil — CNRPark-EXT'deki performans modelin gerçek dünyada ne kadar tutarlı olduğunu gösteriyor.

PS2.0 (Tongji Üniversitesi, Çin, 2018)

600×600 piksel çözünürlüğünde 12.165 surround-view görüntü içeriyor. İç mekan, açık hava gün ışığı, sokak lambası, gölge, yağmur ve eğimli zemin olmak üzere altı farklı test kategorisi var. Ama önemli bir eksik: sadece park yeri tespitini kapsıyor, doluluk bilgisi sunmuyor.

Veri Seti	Görüntü	Park Slotu	Hava Koşulu	Gece	Doluluk
PKLot	12.417	695.900	Var	Yok	Var
CNRPark-EXT	~150.000 patch	164	Var	Var	Var
PS2.0	12.165	—	Var	Var	Yok

2.8 Sensör Tabanlı vs Kamera Tabanlı Sistemler

Piyasada iki farklı yaklaşım yarışıyor.

Sensör Tabanlı Sistemler

Manyetik sensörler: Araç üzerinden geçtiğinde dünya manyetik alanında oluşan değişimi algılıyor. Her park slotu için ayrı sensör gerekiyor, yüksek doğruluk (%98-99) sağlıyor. Ayrıca sensörlerin zemine gömülmesi ve kablolama gerekliliği, büyük otoparklarda kurulum maliyetini artırıyor ve bakım süreçlerini karmaşık hâle getiriyor.

Ultrasonik sensörler: Yüksek frekanslı ses dalgaları ile mesafe ölçüyor. Kurulumu daha kolay ama yağmur ve rüzgarda güvenilirlik düşüyor. Ayrıca her park yerine ayrı sensör yerleştirilmesi gerektiği için büyük otoparklarda maliyet ve bakım ihtiyacı hızla artabiliyor.

Döngü bobinleri (Loop Coil): Yola gömülen metal spiral bobin, araç metalini tespit ediyor. Çok yüksek doğruluk (%98-99) ama kurulum yol kazma gerektiriyor — en pahalı ve en yerleştirmesi zor yöntem. Bunun yanında altyapıya gömülü olduğu için arıza durumunda onarım süreci zor ve trafik açısından kesinti gerektirebiliyor.

Kamera Tabanlı Sistemler

Tek kamera birden fazla park yerini izleyebiliyor. Ek altyapı gerektirmiyor, mevcut güvenlik kameralarıyla çalışabiliyor. Hava koşullarına karşı daha hassas ama yazılım iyileştirmeleriyle bu kısmen telafi edilebiliyor.

Kritik avantaj — Elektrikli Araç Uyumu: Manyetik sensörlerin gelecekteki en büyük kısıtı elektrikli araçlar. Elektrikli araçlar çok hafif yapıları ve verimli gövde tasarımları nedeniyle manyetik alana çok az etki ediyor. Bu durum, yalnızca manyetometreye dayalı park sensörlerini EV'ler karşısında yetersiz kılıyor. Kamera tabanlı sistemler bu sorunu hiç yaşamıyor — araç elektrikli mi dizel mi olduğu fark etmiyor, görüntüde görülüyor.

Kriter	Ultrasonik	Manyetik	Döngü Bobin	Kamera (CV)
Doğruluk	%95-98	%90-97	%98-99	%92-99
Kurulum	Kolay	Kolay	Zor (yol kazıma)	Orta
Bakım	Düşük	Düşük	Orta	Orta
Kapsam	1 yer/sensör	1 yer/sensör	1-2 yer	Çoklu yer
Hava koşulları	Orta	İyi	İyi	Zayıf-Orta
EV uyumu	İyi	Zayıf	İyi	Mükemmel
Ölçeklenebilirlik	Düşük	Düşük	Çok Düşük	Yüksek
Birim maliyet	Düşük	Düşük	Yüksek	Paylaşımlı (düşük)

Akademik çalışmaların yanı sıra park yönetimi alanında faaliyet gösteren ticari sistemler de mevcuttur. Bu sistemlerin incelenmesi, geliştirilen projenin gerçek dünya uygulamalarıyla nasıl ilişkilendirildiğini ortaya koymak açısından önem taşımaktadır.

2.8.1 Parquery (İsviçre, 2014 — ETH Zürich Çıkışlı)

ETH Zürich araştırmacıları tarafından kurulan Parquery [11], mevcut kameralardan alınan görüntüleri yapay zeka ile işleyerek araçları tespit etmektedir. Özel donanım gerektirmemekte; var olan kamera altyapısına bir yazılım katmanı eklenerek sisteme dahil edilebilmektedir.

Yalnızca doluluk tespiti değil; yasak bölgede park, engelli parkının işgali, süreli park süresinin aşılması ve şarj istasyonunun yanlış kullanımı gibi ihlaller de gerçek zamanlı olarak tespit edilebilmektedir. Bu özellikler açısından Parquery, geliştirilen projeyle en çok örtüşen ticari sistem konumundadır.

- Ölçek: 30'dan fazla ülke, 100'den fazla aktif kurulum
- İddia edilen doğruluk: %99
- İş modeli: SaaS (yazılım hizmet aboneliği), park yeri başına fiyatlandırma
- Teknik yaklaşım: Kamera tabanlı, ek altyapı gerektirmez

2.8.2 Quercus Technologies (İspanya, 25+ Yıl)

Quercus Technologies [12], "sanal döngü" adını verdikleri teknoloji ile zemine gömülü sensör kullanmaksızın araç varlığını tespit etmektedir. Sistem, gölge ve ani ışık değişimi gibi dış etkenleri minimize edecek biçimde tasarlanmıştır. Şirketin geliştirdiği QAI (Quercus AI) platformu aynı anda birden fazla park alanında plaka tanıma ve doluluk tespiti yapabilmektedir.

- Ölçek: 60'tan fazla ülkede 11.000'den fazla kurulum
- Plaka tanıma: 150'den fazla ülke plakasını desteklemekte, %99 ve üzeri doğruluk
- Teknik yaklaşım: Kamera tabanlı görüntü işleme + isteğe bağlı sensör entegrasyonu
- Öne çıkan özellik: Gece ve kötü hava koşullarında yüksek performans

2.8.3 Metropolis (ABD, 2012)

Metropolis, otopark sektörünün en yüksek fonlamasını almış şirketi olma özelliğini taşımaktadır (1,7 milyar dolar). Bilgisayarlı görme tabanlı ödeme sistemleri geliştirmektedir: araç plakası kamerayla tanınmakta ve otomatik olarak ücretlendirme gerçekleştirilmektedir. Bu sayede gişe, ödeme makinesi veya kart okuyucuya ihtiyaç kalmamaktadır.

- Ölçek: ABD'de büyük otopark operatörleriyle yaygın entegrasyon
- Teknik yaklaşım: Plaka tanıma (LPR — License Plate Recognition)
- İş modeli: Otopark operatörleriyle gelir paylaşımı
- Öne çıkan özellik: Temassız ödeme deneyimi, insan operatörü ortadan kaldırma

2.8.4 ParkHub + Smarking (ABD)

ParkHub ve Smarking, büyük etkinlik mekanları ve stadyumlara yönelik SaaS tabanlı park yönetimi çözümleri sunmaktadır. 2.500'den fazla park lokasyonunu kapsayan platformda gerçek zamanlı veri erişimi, dinamik fiyatlandırma ve dijital izin yönetimi temel özellikler arasında yer almaktadır.

- Ölçek: 2.500'den fazla lokasyon
- Teknik yaklaşım: IoT sensörleri + veri analitiği

- İş modeli: SaaS aboneliği
- Öne çıkan özellik: Talebe dayalı dinamik fiyatlandırma (konser, maç vb. etkinliklerde anlık fiyat ayarı)

2.8.5 IEM (Almanya) — Intelligent Parking Solutions

IEM, Avrupa'nın en köklü park yönetimi şirketlerinden biridir. Hem sensör tabanlı hem de kamera tabanlı çözümler sunmaktadır. Özellikle büyük alışveriş merkezi ve hastane otoparkları için kapsamlı sistemler kurulmaktadır. Şehir genelinde merkezi yönetim paneli üzerinden tüm park alanlarının anlık durumu izlenebilmektedir.

- Teknik yaklaşım: Ultrasonik sensör + kamera hibrit
- Öne çıkan özellik: Merkezi yönetim paneli, şehir ölçekli entegrasyon

2.8.6 Bosch (Almanya) — Çok Katmanlı Park Çözümleri

Bosch, park yönetimi alanında birbirinden farklı üç yaklaşımla faaliyet yürütmektedir.

Community-Based Parking: Araçların sensörlerinden ve akıllı şehir altyapısından toplanan veriler bulut tabanlı yapay zeka platformuna aktarılmaktadır. Bu verilerden gerçek zamanlı dijital park haritası oluşturulmakta; sürücüler müsait park yeri bilgisine navigasyon uygulamaları üzerinden erişebilmektedir.

Automated Valet Parking (AVP): Stuttgart Havalimanı ve Almanya'daki 15 farklı otogarajda hayata geçirilmiştir. Araç, sürücüsüz olarak otopark girişinden boş alana kadar yönlendirilmektedir. Sistem LiDAR, kamera ve radar sensörlerini bir arada kullanmaktadır.

INTEOX Kameralar: Edge AI (uç bilişim) mimarisiyle çalışan bu kameralar, görüntü işlemeyi kamera üzerinde gerçekleştirmektedir. Detroit M-1 koridorunda konuşlandırılmış olup gerçek zamanlı park durumu tespiti yapılmaktadır. Buluta yalnızca sonuç verisi gönderilmekte; bu sayede bant genişliği ve gecikme sorunları en aza indirilmektedir.

- Teknik yaklaşım: Sensör füzyonu + kamera + edge AI
- Öne çıkan özellik: Otomatik vale park, edge AI kamera altyapısı
- Ölçek: Almanya genelinde 15+ otogaraj, ABD Detroit koridoru

2.8.7 Hikvision (Çin) — Büyük Ölçekli AI Kamera Sistemleri

Hikvision, dünya genelinde en yaygın kullanılan güvenlik kamerası üreticileri arasında yer almakta ve park yönetimi için özelleştirilmiş yapay zeka çözümleri sunmaktadır.

Guanlan Büyük AI Modeli: Çok sayıda kameranın ürettiği görüntüyü merkezi yapay zeka modeliyle analiz etmekte ve park doluluk haritası çıkarmaktadır. DeepinViewX serisi kameralar ile yüksek hassasiyetli araç tespiti ve plaka tanıma yapılmaktadır.

Irida Labs İş Birliği: Avrupa merkezli Irida Labs ile gerçekleştirilen ortak projede uç bilişim (edge vision AI) mimarisi kullanılmaktadır. Hikvision'ın kamera donanımı ile Irida Labs'ın görüntü işleme yazılımı birleştirilmekte; görüntü analizi kamera üzerinde tamamlanmaktadır.

- Teknik yaklaşım: Kamera tabanlı derin öğrenme + ANPR (plaka tanıma)
- Öne çıkan özellik: Büyük ölçekli merkezi AI modeli, edge + merkezi hibrit mimari

2.8.8 Dahua Technology (Çin) — WizMind AI Platformu

Dahua Technology, WizMind yapay zeka platformunu park yönetimi alanında aktif olarak kullanmaktadır.

Danimarka'daki havalimanı otopark projesinde WizMind kameralar konuşlandırılmıştır. Sistem araç türünü, rengini ve plakasını eş zamanlı olarak tespit edebilmekte; %95 ve üzeri plaka tanıma doğruluğu bildirilmektedir. Boş ve dolu park yerleri anlık olarak izlenmekte, veriler merkezi yönetim sistemine aktarılmaktadır.

- Teknik yaklaşım: Kamera tabanlı + ANPR
- Öne çıkan özellik: Araç türü + renk + plaka eş zamanlı tespiti, %95+ plaka doğruluğu
- Referans kurulum: Danimarka havalimanı otoparkı

2.8.9 Frogparking (Yeni Zelanda) — Hibrit Sensör Çözümü

Frogparking, açık hava park alanları için ultrasonik sensör ile kamera teknolojisini birleştiren hibrit bir yaklaşım benimsemektedir. Kapalı otoparklar yerine açık hava parkında karşılaşılan hava koşulları ve ışık değişkenliği sorunlarını gidermek amacıyla çift kaynaklı algılama tercih edilmektedir.

- Teknik yaklaşım: Ultrasonik sensör + kamera hibrit
- Öne çıkan özellik: Açık hava park alanları için optimize edilmiş, hava koşullarına dayanıklı
- Ölçek: Yeni Zelanda ve Avustralya merkezli, uluslararası expansion sürecinde

Ticari Sistemlerin Karşılaştırması

Sistem	Ülke	Yaklaşım	Plaka Tanıma	İhlal Tespiti	Ek Donanım
Parquery	İsviçre	Kamera (AI)	Hayır	Evet	Hayır
Quercus	İspanya	Kamera + Sensör	Evet	Kısmi	İsteğe bağlı
Metropolis	ABD	Kamera (LPR)	Evet	Hayır	Hayır
ParkHub	ABD	IoT Sensör	Hayır	Hayır	Evet

Sistem	Ülke	Yaklaşım	Plaka Tanıma	İhlal Tespiti	Ek Donanım
IEM	Almanya	Sensör + Kamera	Kısmi	Hayır	Evet
Bosch	Almanya	Sensör + Edge AI	Evet	Hayır	Evet (AVP)
Hikvision	Çin	Kamera (ANPR+AI)	Evet	Kısmi	Hayır
Dahua	Çin	Kamera (WizMind)	Evet	Hayır	Hayır
Frogparking	Yeni Zelanda	Sensör + Kamera	Hayır	Hayır	Evet
Geliştirilen Proje	Türkiye	Kamera (YOLOv8)	Hayır	Altyapı hazır	Hayır

2.9 Projenin Literatürdeki Yeri

Tüm bu çalışmalar incelendikten sonra geliştirilen projenin literatürdeki konumu net biçimde ortaya konabilmektedir.

Güçlü Yönler

Güncel model kullanılmaktadır: 2017-2021 arasındaki çalışmalar AlexNet, VGG, mAlexNet gibi daha eski nesil CNN modelleriyle yürütülmüştür. Geliştirilen projede YOLOv8 kullanılmaktadır; bu model söz konusu alternatiflerin büyük çoğunluğundan daha hızlı ve daha doğrudur.

Poligon tabanlı park alanı analizi altyapısı geliştirilmiştir: Literatürdeki çalışmaların büyük çoğunluğu yalnızca "dolu/boş" ikili sınıflandırma yapmaktadır. Geliştirilen projede park alanları ve yasak bölgeler poligon olarak tanımlanabilmekte; IoU hesabıyla doluluk analizi yapılabilmektedir. Üç sınıflı çıktı (park edilebilir / dolu / yasak) ilerleyen aşamada arayüze entegre edilmesi planlanmaktadır.

Sentetik veri üretilebilmektedir: Etiketli gerçek veri ihtiyacını azaltmak için otomatik sentetik veri üretimi yapılmaktadır. Bu yaklaşım hem iş yükünü azaltmakta hem de veri çeşitliliğini artırmaktadır.

Ek donanım gerektirmemektedir: Mevcut kamera altyapısıyla çalışılabilmekte; Parquery gibi ticari sistemlerle aynı felsefe benimsenmiştir.

Zayıf Yönler

PKLot ile kapsamlı karşılaştırma yapılmamıştır: Literatürdeki standart benchmark üzerinde henüz test gerçekleştirilmemiştir.

Araç takibi bulunmamaktadır: Geçen araç ile park edilmiş araç henüz ayırt edilememektedir. ByteTrack entegrasyonu ilerleyen aşamalarda planlanmaktadır.

Park alanları manuel tanımlanmaktadır: APSD-OC ve PakLoc gibi çalışmalar bunu otomatik gerçekleştirmektedir; geliştirilen projede henüz el ile çizim yöntemi kullanılmaktadır.

Gece ve yağmur testi gerçekleştirilmemiştir: Literatürdeki çalışmaların bir kısmı farklı hava koşullarında test yapmaktadır; bu testler henüz gerçekleştirilememiştir.

Temel Alınan Çalışmalar

Geliştirilen projeye en yakın referans da Luz vd. (2024) [7] çalışmasıdır; YOLOv8 + ROI analizi kombinasyonu bu proje ile büyük ölçüde örtüşmektedir.

Ticari referans olarak Parquery [11] baz alınmaktadır: mevcut kameraya yapay zeka ekleyerek ek altyapı gerektirmeden analiz yapılması hedeflenmektedir.

Gelecekteki geliştirme için ise APSD-OC otomatik park yeri tespiti yaklaşımı ve ByteTrack araç takip algoritması referans noktalarımız.

2.10 Literatür Özeti Tablosu

Çalışma	Yıl	Yöntem	Veri Seti	Doğruluk	Bizimle Fark
AlexNet park uyarlaması	2015	CNN patch	PKLot	~%95	Eski model, sadece dolu/boş
mAlexNet (Amato vd.)	2017	Küçük CNN	CNRPark	~%96	Hafif ama sınırlı
CarNet (Nurullayev & Lee)	2019	Dilated CNN	PKLot+CNR	~%97	Daha iyi ama büyük

Çalışma	Yıl	Yöntem	Veri Seti	Doğruluk	Bizimle Fark
MobileNetV3+CBAM (Yuldashev vd.)	2023	Lightweight CNN	PKLot+CNR	%98.01	Sınıflandırma odaklı, ihlal yok
APSD-OC (Grbić & Koch)	2023	YOLOv5+ResNet34	PKLot+CNR	Yüksek	Otomatik slot tespiti var
da Luz vd.	2024	YOLOv8-v11+ROI	Özel	%99.68	En yakın referans, ihlal yok
Bizim Sistemimiz	2025	YOLOv8+IoU+Kural	Sentetik+Gerçek	Ölçülecek	3 sınıf + ihlal tespiti

3. Sistem Mimarisi

Bu bölümde geliştirilen sistemin nasıl çalıştığı, hangi parçalardan oluştuğu ve bu parçaların birbiriyle nasıl etkileşime girdiği anlatılmaktadır.

3.1 Genel Bakış

Geliştirilen sistem temelde şu soruyu yanıtlamaktadır: "Bu kamera görüntüsündeki araç, orada park edebilir mi?"

Bu soruyu yanıtlamak için sırayla dört işlem gerçekleştirilmektedir: görüntüdeki araçlar bulunmakta, park alanı bilgisi yüklenmekte, araçların bu bölgelerle örtüşüp örtüşmediği hesaplanmakta ve karar ekranda gösterilmektedir. Bu yapıya pipeline adı verilmektedir.

Kamera / Video / Resim

↓

[YOLOv8 ile Araç Tespiti]

→ Bounding box, güven skoru, araç tipi

↓

[Park Alanı Poligonları Yükleniyor]

→ Park edilebilir ve yasak bölgeler

↓

[IoU Hesabı]

→ Her araç için bölge örtüşme oranı

↓

[Karar Motoru]

→ Park Edilebilir / Park Alanı Dolu / Park Edilemez

↓

[PyQt5 Arayüzü]

→ Gerçek zamanlı görselleştirme

3.2 Bileşen 1: Araç Tespiti (VehicleDetector)

Dosya: src/detection/vehicle_detector.py

Görüntü YOLOv8 modeline verilmekte, model her araç için bounding box ve güven skoru döndürmekte, yalnızca araç sınıfları filtrelenmektedir.

COCO mu, Fine-Tuned Model mi?

Sistem iki farklı modelle çalışabilmektedir. COCO ve Fine-tuned. COCO önceden eğitilmiş model 80 nesneyi tanımakta, araç sınıfları sonradan filtrelenmektedir (ID: 2, 3, 5, 7). Fine-

tuned model ise yalnızca 4 araç sınıfını 0'dan başlayan ID'lerle tanımaktadır. Sistem model adına bakarak doğru ID haritasını otomatik seçmektedir:

Model adında "fine_tuned" geçiyor mu?

Evet → class_map = {0: "car", 1: "motorcycle", 2: "bus", 3: "truck"}

Hayır → class_map = {2: "car", 3: "motorcycle", 5: "bus", 7: "truck"}

3.3 Bileşen 2: Park Alanı Etiketleme (ZoneAnnotator)

Dosya: src/parking/zone_annotator.py

Bir fotoğraf açılmakta, fareyle köşe noktaları işaretlenmekte ve program bu noktalardan poligon oluşturmaktadır. Kaydedilen bilgi JSON formatında dosyaya yazılmaktadır.

Tuş	İşlev
Sol tık	Nokta ekle
ENTER / S	Park alanı kaydet
F	Yasak bölge kaydet
Z	Son noktayı geri al
C	Çizimi temizle
D	Son bölgeyi sil
Q / ESC	Kaydet ve çık

Literatürde otomatik park yeri tespiti çalışmaları mevcuttur (APSD-OC [9]); ancak her park alanının şekli ve çizgi düzeni farklıdır. Manuel çizim bir kez yapılmakta, sonrasında sistem otomatik çalışmaktadır. Parquery [11] gibi ticari sistemler de benzer yaklaşımı benimsemektedir.

3.4 Bileşen 3: IoU Hesabı (ZoneLoader)

Dosya: src/parking/zone_loader.py

IoU (Intersection over Union), iki şeklin örtüşme oranını 0-1 arası bir değerle ifade etmektedir:

$$\text{IoU} = \frac{[\text{Kesişim Alanı}]}{[\text{Birleşim Alanı}]}$$

Park alanları poligon şeklinde olduğundan piksel maskesi yaklaşımı kullanılmaktadır: araç bounding box ve park poligonu ayrı piksel maskelerine çizilmekte, mantıksal AND ile kesişim, OR ile birleşim alanı hesaplanmaktadır. IoU 0.3 eşiğini aşarsa araç o bölgeyle ilişkilendirilmektedir. Eşiğin 0.5 yerine 0.3 seçilmesinin nedeni kamera açısı ve poligon çizim toleransıdır.

3.5 Bileşen 4: Karar Motoru

Bir araç için:

1. Yasak bölgelerle IoU > 0.3 → "PARK EDİLEMEZ"
2. Park alanlarıyla IoU > 0.3 → "PARK ALANI DOLU"
3. Eşleşme yok → "Serbest alan"

Park alanı için araç yoksa → "PARK EDİLEBİLİR"

Mevcut çalışmaların büyük çoğunluğu yalnızca "dolu/boş" ikili sınıflandırması yapmaktadır. Geliştirilen altyapıda üçüncü sınıf (yasak bölge) da desteklenmektedir; bu özellik ilerleyen aşamada arayüze entegre edilmesi planlanmaktadır.

3.6 Bileşen 5: Sentetik Veri Üretici

Dosya: scripts/synthetic_data_generator.py

Modeli park senaryosuna özel eğitmek için etiketli veriye ihtiyacımız var. Gerçek park alanı görüntüsünü etiketlemek çok zaman alıyor. Bunun yerine bilgisayarda otomatik görüntüler üretiyoruz. Her görüntüde gri bir zemin üzerine 1-5 arası rastgele araç yerleştirilmektedir. Her araç tipi farklı renk ve basit detaylarla temsil edilmektedir. YOLO formatında etiket dosyaları otomatik oluşturulmaktadır. 1000 görüntünün 800'ü eğitime, 200'ü doğrulamaya ayrılmıştır.

3.7 Bileşen 6: YOLOv8 Fine-Tuning

Dosya: scripts/fine_tune_yolo.py

COCO ile eğitilmiş YOLOv8'in öğrendiği genel özellikler korunarak yalnızca park senaryosuna özgü ayrıntılar yeniden öğretilmektedir. Eğitim parametreleri:

Parametre	Değer
Epoch	30
Batch size	8
Görüntü boyutu	640×640
Early stopping	10 epoch

Parametre	Değer
Device	GPU (CUDA) / CPU otomatik

3.8 Bileşen 7: Arayüz (MainWindow)

Dosya: src/ui/main_window.py

PyQt5 ile geliştirilen arayüz kamera, video ve resim kaynaklarını desteklemektedir. Sol tarafta gerçek zamanlı video alanı, sağ tarafta araç tipi sayaçları, FPS göstergesi ve kontrol butonları yer almaktadır. QTimer her 30 ms'de bir kare okuyarak arayüzü bloklamadan güncellemektedir.

Araç Tipi	Çerçeve Rengi
Araba	Yeşil
Motosiklet	Turuncu
Otobüs	Kırmızı
Kamyon	Mor

4. Kullanılan Teknolojiler

Bu bölümde projede kullanılan araçlar, kütüphaneler ve bunların seçilme gerekçeleri açıklanmaktadır.

4.1 Programlama Dili: Python 3.12

PyTorch, OpenCV, Ultralytics ve scikit-learn gibi yapay zeka ve görüntü işleme kütüphaneleri Python'da en iyi desteğe sahiptir. Birden fazla kişinin çalıştığı projede sözdiziminin sade ve okunabilir olması da tercih nedeni olmuştur.

4.2 Nesne Tespiti: YOLOv8 (Ultralytics)

YOLO "You Only Look Once" kelimelerinin baş harflerinden oluşmaktadır. Görüntüye tek seferinde bakarak hem nesnenin ne olduğunu hem de nerede olduğunu aynı anda tespit etmektedir. Bu sayede iki aşamalı yöntemlere (Faster R-CNN gibi) kıyasla çok daha hızlı çalışmaktadır.

Özellik	YOLOv8	Faster R-CNN	SSD
Hız	Çok yüksek	Düşük	Yüksek
Doğruluk	Yüksek	Çok yüksek	Orta
Gerçek zamanlı	Evet	Hayır	Evet
Kullanım kolaylığı	Çok kolay	Zor	Orta

Faster R-CNN doğrulukta öne geçebilmektedir; ancak gerçek zamanlı çalışmamaktadır. Literatürdeki güncel çalışmalar da park tespitinde YOLOv8 kullanmaktadır [7].

Neden Nano (n) Versiyonu?

GTX 1050 Ti Mobile 4GB VRAM'de YOLOv8m ve üzeri modeller gerçek zamanlı çalışamamaktadır. YOLOv8n ise 30+ FPS'i rahatlıkla vermektedir.

Model	mAP	Hız (ms)	Parametre
YOLOv8n	37.3	0.80	3.2M
YOLOv8s	44.9	1.20	11.2M
YOLOv8m	50.2	3.53	25.9M

Transfer Learning

Microsoft COCO veri setiyle [14] önceden eğitilmiş YOLOv8n modeli transfer learning yöntemiyle kullanılmıştır. COCO 330.000'den fazla görüntü ve 80 sınıf içermektedir. Bu ağırlıklar üzerine sentetik veriyle araç tespitine özgü ince ayar (fine-tuning) yapılmıştır [2].

4.3 Görüntü İşleme: OpenCV [15]

Kamera/video okuma, görüntü üzerine çizim, piksel maskesi oluşturma ve park alanı etiketleme aracındaki mouse etkileşimi OpenCV ile gerçekleştirilmektedir. PIL/Pillow video okuma ve gerçek zamanlı işlem için OpenCV kadar kapsamlı değildir.

4.4 Derin Öğrenme Altyapısı: PyTorch 2.2.2 [16]

YOLOv8'in resmi implementasyonu PyTorch tabanlıdır. Doğrudan kullanılmamakta; Ultralytics bu katmanı soyutlamaktadır. Ancak PyTorch yüklü olmadan sistem çalışamamaktadır. TensorFlow tercih edilseydi farklı bir YOLO versiyonu kullanmak gerekirdi.

4.5 Görsel Arayüz: PyQt5 5.15+

Qt, C++ ile yazılmış güçlü bir arayüz framework'üdür. PyQt5 Python'dan Qt'yi kullanmayı sağlamaktadır. Tkinter gerçek zamanlı video gösterimi için yavaş kalmakta, PySimpleGUI ise karmaşık layout yönetiminde yetersiz kalmaktadır. Ayrıca QTimer ile gerçek zamanlı video döngüsü arayüzü bloklamadan çalışmaktadır

4.6 Model Değerlendirme: scikit-learn 1.3+

Eğitim ve test sonuçlarından Accuracy, Precision, Recall, F1-Score ve Confusion Matrix hesaplanmaktadır. Yalnızca accuracy değerine bakmak yanıltıcı olabilmektedir; veri setinin büyük çoğunluğu tek bir sınıfa aitse model tüm örnekleri o sınıfa atasaydı bile yüksek doğruluk elde edebilirdi. Precision, Recall ve F1-Score bu tür sorunları açığa çıkarmaktadır. K-Fold Cross Validation ile modelin gerçek performansı daha güvenilir biçimde ölçülmektedir.

4.7 Veri İşleme ve Görselleştirme

NumPy 1.26.4: IoU hesabında piksel maskeleri NumPy dizileri olarak işlenmektedir.

Pandas 2.0+: Eğitim loglarını ve metrik sonuçlarını tablo halinde düzenlemek için kullanılmaktadır.

Matplotlib 3.7+: Loss eğrileri ve mAP grafikleri çizilmektedir.

Seaborn 0.12+: Confusion matrix ısı haritası olarak görselleştirilmektedir.

4.8 Teknoloji Özet Tablosu

Teknoloji	Versiyon	Kullanım Amacı
Python	3.12	Ana programlama dili
Ultralytics YOLOv8	8.4.24	Araç tespiti

Teknoloji	Versiyon	Kullanım Amacı
PyTorch	2.2.2	YOLOv8 derin öğrenme altyapısı
OpenCV	4.x	Görüntü işleme, video okuma
PyQt5	5.15+	Masaüstü arayüzü
scikit-learn	1.3+	Model değerlendirme metrikleri
NumPy	1.26.4	Sayısal hesaplama
Pandas	2.0+	Veri analizi, log kayıtları
Matplotlib	3.7+	Grafik çizimi
Seaborn	0.12+	Confusion matrix görselleştirme
PyYAML	6.0+	Konfigürasyon dosyası

5. Uygulama ve Bulgular

Bu bölümde projenin şu ana kadar hangi aşamalardan geçtiği ve elde edilen sonuçlar aktarılmaktadır. Proje hâlâ devam ettiği için bazı metrik sonuçları ilerleyen haftalarda güncellenecektir.

5.1 Geliştirme Süreci

Hafta 1 — Temel Kurulum ve Baseline: Klasör yapısı oluşturulmuş, kütüphaneler kurulmuş, VehicleDetector sınıfı yazılmış ve YOLOv8n ilk kez COCO ağırlıklarıyla çalıştırılmıştır. Park alanı etiketleme aracı (ZoneAnnotator) geliştirilmiş, komut satırı test betiği tamamlanmıştır. İlk testte model bir otobüs görüntüsünde %93 güven skoru ile başarılı tespit yapmıştır.

Hafta 2 — Sentetik Veri ve Arayüz: 1000 görüntülük sentetik veri üretilmiş, YOLOv8 fine-tuning betiği hazırlanmış ve PyQt5 tabanlı görsel arayüz geliştirilmiştir. Araç tespiti sonuçları arayüze entegre edilmiş; araç tipi sayaçları, renk kodlaması ve FPS göstergesi eklenmiştir.

5.2 Baseline Sistem Sonuçları

COCO ağırlıklarıyla çalışan YOLOv8n modeli [2] park senaryosuna özel herhangi bir eğitim yapılmadan test edilmiştir. Güven skorları genellikle %80-95 arasında ölçülmüştür. Büyük araçlar (otobüs, kamyon) daha güvenilir tespit edilmiş; kısmen kapalı araçlarda ve olumsuz kamera açılarında güven skoru düşmüştür. Motosikletler küçük olduğu için zaman zaman kaçırılmıştır.

5.3 Sentetik Veri Üretimi

Parametre	Değer
Toplam görüntü	1000
Eğitim seti	800 (%80)
Doğrulama seti	200 (%20)
Görüntü boyutu	640×480 px
Araç/görüntü	1-5 (rastgele)

Her araç tipi basit renkli dikdörtgen ve detaylarla temsil edilmektedir. YOLO formatında normalize edilmiş etiket dosyaları otomatik oluşturulmaktadır.

5.4 Fine-Tuning Sonuçları

Fine-tuning GPU desteğiyle tamamlanmıştır. Early stopping 40. epoch'ta devreye girmiştir.

Metrik	Değer
mAP@0.5	%99.5
En iyi model	runs/detect/models/fine_tuned/yolov8_fine_tuned2/weights/best.pt

5.5 Görsel Arayüz

Geliştirilen PyQt5 arayüzü kamera akışı, video oynatma ve tek resim analizini desteklemektedir. Karanlık tema (Fusion stili), gerçek zamanlı araç sayaçları, FPS göstergesi ve durum çubuğu içermektedir.

GTX 1050 Ti Mobile donanımında ölçülen yaklaşık FPS değerleri:

Kaynak	Çözünürlük	Yaklaşık FPS
Video (720p)	1280×720	~18-22
Video (480p)	854×480	~25-30
Resim	Değişken	Anlık

5.6 IoU Tabanlı Doluluk Analizi

Park alanı JSON dosyası yüklendiğinde sistem araç tespiti sonuçlarını bu alanlarla karşılaştırmaktadır.

Durum	IoU (park)	IoU (yasak)	Karar
Park alanı dolu	> 0.3	< 0.3	Park Alanı Dolu
Yasak ihlali	< 0.3	> 0.3	Park Edilemez
Eşleşme yok	< 0.3	< 0.3	Serbest alan
Park alanı boş	Araç yok	—	Park Edilebilir

IoU pipeline kodu çalışır durumdadır; arayüze entegrasyonu ilerleyen aşamada tamamlanması planlanmaktadır.

5.7 Mevcut Sistem Sınırlamaları

Araç takibi yok: Geçen bir araç geçici olarak "Dolu" kararı tetikleyebilmektedir. ByteTrack entegrasyonu planlanmaktadır.

Gece ve yağmur testi yapılmadı: Veri artırma teknikleriyle (parlaklık değişimi, bulanıklaştırma) bu kısmen telafi edilecektir.

Sentetik veri gerçek değil: Gerçek kamera görüntüleriyle doku ve ışık farklılıkları mevcuttur. PKLot veri setiyle [8] ek eğitim planlanmaktadır.

6. SONUÇ VE GELECEK ÇALIŞMALAR

6.1 Genel Değerlendirme

Bu projede kamera görüntülerinden araç tespiti ve park uygunluğu analizi yapan yapay zeka tabanlı bir sistem geliştirilmesi hedeflenmiştir. İlk iki haftada gerçekleştirilen çalışmalar şu şekilde özetlenebilir:

- YOLOv8 tabanlı araç tespit sistemi çalışır duruma getirilmiştir.
- Park alanı etiketleme aracı (ZoneAnnotator) tamamlanmıştır.
- IoU tabanlı doluluk analizi altyapısı hazırlanmıştır.
- 1000 görüntülük sentetik veri üretilmiş ve fine-tuning tamamlanmıştır (mAP@0.5: %99.5).
- PyQt5 arayüzü tamamlanmış; kamera, video ve resim üzerinde gerçek zamanlı analiz yapılabilmektedir.

Başlangıçta belirlenen hipotez — "YOLOv8 ile poligon tabanlı park analizinin birleştirilmesi, park uygunluğunu üç sınıfta %85 ve üzeri doğrulukla sınıflandırmak için yeterlidir" — henüz tam olarak test edilememiştir. Kurulan altyapı bu testi mümkün kılmaktadır.

6.2 Literatürle Karşılaştırma

2017'deki mAlexNet [3] ve 2019'daki CarNet [4] gibi çalışmalara kıyasla çok daha güncel bir model (YOLOv8) kullanılmaktadır. Söz konusu çalışmalar yalnızca "dolu/boş" kararı üretirken geliştirilen projede "yasak bölge" sınıfı da eklenmiştir.

da Luz vd.'nin 2024 çalışması [7] mevcut projeye en yakın referanstır; YOLOv8 + ROI analizi kombinasyonu büyük ölçüde örtüşmektedir. Ticari tarafta ise Parquery [11] ile aynı felsefe paylaşılmaktadır: mevcut kameraya yapay zeka ekle, ek altyapı gerektirme.

6.3 Karşılaşılan Zorluklar

PyTorch/NumPy sürüm uyumsuzluğu: PyTorch 2.10 kurulduğunda DLL hatası alınmıştır. PyTorch 2.2.2 ve NumPy 1.26.4 kombinasyonuna geçilerek sorun giderilmiştir.

GPU kullanılamaması: Başlangıçta sürücü uyumsuzluğu nedeniyle CPU kullanılmıştır. CUDA 12.1 destekli PyTorch kurulumunun ardından GPU aktif hale getirilmiştir.

Class ID uyumsuzluğu: COCO modelinde araç ID'leri 2, 3, 5, 7 iken fine-tuned modelde 0, 1, 2, 3'tür. VehicleDetector'a model adına göre otomatik seçim eklenerek sorun çözülmüştür.

Timer çoklu bağlantı: Arayüzde her buton tıklamasında timer'a yeni bağlantı eklenmekteydi. Timer başta bir kez bağlanacak şekilde yeniden düzenlenmiştir.

KAYNAKLAR

[1] Spherical Insights, (2024), Smart Parking Systems Market Report, https://www.sphericalinsights.com

[2] Jocher, G. v.d., (2023), "Ultralytics YOLOv8", https://github.com/ultralytics/ultralytics

[3] Amato, G. v.d., (2017), "Deep learning for decentralized parking lot occupancy detection", Expert Systems with Applications, Vol.72, pp.327-334.

[4] Nurullayev, S. ve Lee, S.W., (2019), "Generalized parking occupancy analysis based on dilated convolutional neural network", Sensors, Vol.19, No.2, s.277.

[5] Almeida, P.R.L. v.d., (2015), "PKLot – A robust dataset for parking lot classification", Expert Systems with Applications, Vol.42, No.11, pp.4937-4949.

[6] Sandler, M. v.d., (2018), "MobileNetV2: Inverted Residuals and Linear Bottlenecks", CVPR 2018, Salt Lake City, pp.4510-4520.

[7] da Luz, G.P.C.P. v.d., (2024), "Smart parking with pixel-wise ROI selection for vehicle detection using YOLOv8, YOLOv9, YOLOv10, and YOLOv11", arXiv:2412.01983.

[8] Yuldashev, B. v.d., (2023), "Parking lot occupancy detection with improved MobileNetV3", Sensors, Vol.23, No.18.

[9] Grbić, R. ve Koch, T., (2023), "APSD-OC: Automatic parking space detection and occupancy classification", Expert Systems with Applications.

[10] Zhang, Y. v.d., (2022), "ByteTrack: Multi-object tracking by associating every detection box", ECCV 2022, Tel Aviv, pp.1-21.

[11] Parquery AG, (2024), Parquery Parking Intelligence, https://parquery.com

[12] Quercus Technologies, (2024), BirdWatch & QAI Systems, https://www.quercus-technologies.com

[13] Xie, X. v.d., (2017), "Real-time illegal parking detection system based on deep learning", arXiv:1710.02546.

[14] Lin, T.Y. v.d., (2014), "Microsoft COCO: Common Objects in Context", ECCV 2014, Zurich, pp.740-755.

[15] OpenCV Documentation, (2024), https://opencv.org

[16] PyTorch Documentation, (2024), https://pytorch.org